

Relational Database

Core Concepts — Examples

Tables · Rows · Columns · Keys · Schema · Index · Constraints · Relationships · ACID
Every concept illustrated with multiple real-world SQL examples

Practical Examples — College · Bank · E-commerce · Hospital scenarios

Relational Databases Study Guide

Table of Contents

- 1. Table (Relation)** — Hospital, Library, E-commerce table examples
- 2. Row / Tuple** — INSERT, UPDATE, DELETE with Employee & Order data
- 3. Column / Attribute** — Defining columns across 3 different real scenarios
- 4. Primary Key** — Surrogate, natural, composite, UUID — 5 examples
- 5. Foreign Key** — Parent-child links with cascade behaviour examples
- 6. Schema** — Complete Bank & College database schema examples
- 7. Index** — Index examples on Employees, Products, Logs tables
- 8. Constraints** — NOT NULL, UNIQUE, CHECK, DEFAULT — 4 examples each
- 9. Relationships** — 1:1 Employee-Passport, 1:N Customer-Orders, M:N Student-Course
- 10. ACID Properties** — Bank transfer, ticket booking, hospital record transactions

1

Table (Relation)

Storing entity data in rows and columns

A table organises all data about one entity. Every table has a name, fixed columns, and zero or more rows. Below are three real-world tables from different domains.

Ex 1.1 Hospital — Patients Table

PatientID	FullName	DateOfBirth	BloodGroup	WardID	AdmissionDate
P001	Ravi Kumar	1985-03-12	O+	W03	2024-06-01
P002	Meena Pillai	1992-07-24	A+	W01	2024-06-03
P003	Suresh Nair	1978-11-05	B-	W02	2024-06-05
P004	Anitha Rao	2000-01-18	AB+	W03	2024-06-07
P005	Kiran Babu	1965-09-30	O-	W01	2024-06-08

```
CREATE TABLE Patients ( PatientID CHAR(4) PRIMARY KEY, FullName VARCHAR(100) NOT NULL,
DateOfBirth DATE NOT NULL, BloodGroup CHAR(3), WardID CHAR(3) REFERENCES Wards(WardID),
AdmissionDate DATE DEFAULT (CURRENT_DATE) );
```

Ex 1.2 Library — Books Table

ISBN	Title	Author	Genre	YearPublished	CopiesAvailable
978-0-061-96436-9	To Kill a Mockingbird	Harper Lee	Fiction	1960	3
978-0-743-27356-5	The Great Gatsby	F. Scott Fitzgerald	Classic	1925	2
978-0-385-33348-1	The Alchemist	Paulo Coelho	Philosophy	1988	5
978-0-679-72020-1	Crime and Punishment	Fyodor Dostoevsky	Classic	1866	1

```
CREATE TABLE Books ( ISBN VARCHAR(20) PRIMARY KEY, Title VARCHAR(200) NOT NULL, Author
VARCHAR(100) NOT NULL, Genre VARCHAR(50), YearPublished YEAR, CopiesAvailable INT
DEFAULT 1 CHECK(CopiesAvailable >= 0) );
```

Ex 1.3 E-commerce — Products Table

ProductID	Name	Category	Price	StockQty	IsActive
101	Wireless Earbuds	Electronics	1299.00	245	TRUE
102	Yoga Mat	Fitness	599.00	89	TRUE
103	Steel Water Bottle	Kitchen	349.00	312	TRUE
104	Bluetooth Speaker	Electronics	2499.00	0	FALSE

```
CREATE TABLE Products ( ProductID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(150) NOT NULL, Category VARCHAR(50), Price DECIMAL(10,2) NOT NULL CHECK(Price > 0), StockQty INT DEFAULT 0 CHECK(StockQty >= 0), IsActive BOOLEAN DEFAULT TRUE );
```

Each row represents one complete entity instance. These examples show inserting, updating, and deleting rows across multiple scenarios.

INSERT Examples — Adding New Rows

Ex 2.1 Insert a single employee

```
INSERT INTO Employees (FirstName, LastName, Email, Department, Salary, HireDate) VALUES ('Kavya', 'Menon', 'kavya.menon@company.com', 'Engineering', 75000.00, '2024-06-01');
```

Ex 2.2 Insert multiple orders at once

```
INSERT INTO Orders (CustomerID, ProductID, Quantity, OrderDate, Status) VALUES (201, 101, 2, '2024-06-10', 'Pending'), (202, 103, 1, '2024-06-10', 'Shipped'), (203, 102, 3, '2024-06-10', 'Delivered'), (201, 104, 1, '2024-06-11', 'Pending');
```

Ex 2.3 Copy graduated students to an Alumni table

```
INSERT INTO Alumni (StudentID, FullName, Email, GraduationYear) SELECT StudentID, FullName, Email, 2024 FROM Students WHERE Status = 'Graduated' AND Batch = 2020; -- Copies all 2020-batch graduates into the Alumni table in one statement
```

UPDATE Examples — Modifying Existing Rows

Ex 2.4 Give a 10% raise to all Engineering staff

```
UPDATE Employees SET Salary = Salary * 1.10, LastModified = CURRENT_TIMESTAMP WHERE Department = 'Engineering'; -- Affected rows: all employees in Engineering dept
```

Ex 2.5 Mark out-of-stock products as inactive

```
UPDATE Products SET IsActive = FALSE WHERE StockQty = 0; -- Before: ProductID 104 (Bluetooth Speaker) StockQty=0, IsActive=TRUE -- After: ProductID 104 StockQty=0, IsActive=FALSE
```

Ex 2.6 Update one patient's ward after transfer

```
UPDATE Patients SET WardID = 'W05', TransferDate = '2024-06-12' WHERE PatientID = 'P003'; -- Ravi Kumar transferred from Ward W02 to ICU Ward W05
```

DELETE Examples — Removing Rows

Ex 2.7 Remove a specific book (no copies left, decommissioned)

```
DELETE FROM Books WHERE ISBN = '978-0-679-72020-1'; -- Permanently removes Crime and Punishment from catalogue
```

Ex 2.8 Delete all Pending orders older than 30 days

```
DELETE FROM Orders WHERE Status = 'Pending' AND OrderDate < DATE_SUB(CURRENT_DATE,  
INTERVAL 30 DAY); -- Cleans up stale unprocessed orders
```

■ ■ **Always test DELETE with SELECT first**

Before running DELETE, run: `SELECT * FROM Orders WHERE Status='Pending' AND OrderDate < DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY);` — verify rows first.

Columns define the type and rules for each piece of data in every row. The examples below show column definitions for three domains and how to change them.

Column Definitions — 3 Domain Examples

Ex 3.1 Employee table — HR domain

```
CREATE TABLE Employees ( EmpID INT PRIMARY KEY AUTO_INCREMENT, -- surrogate key
  FirstName VARCHAR(50) NOT NULL, -- required LastName VARCHAR(50) NOT NULL, Email
  VARCHAR(100) UNIQUE NOT NULL, -- must be unique Phone VARCHAR(15), -- optional
  Department VARCHAR(60) NOT NULL DEFAULT 'General', -- defaults to General Salary
  DECIMAL(12,2) NOT NULL CHECK(Salary >= 15000), HireDate DATE NOT NULL DEFAULT
  (CURRENT_DATE), IsActive BOOLEAN DEFAULT TRUE, PhotoURL VARCHAR(300) -- nullable URL );
```

Ex 3.2 Bank Accounts table — Finance domain

```
CREATE TABLE Accounts ( AccountNo CHAR(12) PRIMARY KEY, -- fixed 12-digit code
  HolderName VARCHAR(100) NOT NULL, AccountType ENUM('Savings','Current','FD','RD') NOT
  NULL DEFAULT 'Savings', -- controlled vocabulary Balance DECIMAL(15,2) NOT NULL DEFAULT
  0.00 CHECK(Balance >= 0), -- cannot go negative OpenedDate DATE NOT NULL, BranchCode
  CHAR(6) NOT NULL, IFSC CHAR(11) NOT NULL, IsActive BOOLEAN DEFAULT TRUE );
```

Ex 3.3 Flight Bookings table — Travel domain

```
CREATE TABLE Bookings ( BookingID INT PRIMARY KEY AUTO_INCREMENT, PNR CHAR(6) UNIQUE NOT
  NULL, -- 6-char booking code PassengerID INT NOT NULL, FlightID INT NOT NULL, SeatNo
  VARCHAR(4), -- e.g. '12A' Class ENUM('Economy','Business','First') DEFAULT 'Economy',
  FareAmount DECIMAL(10,2) NOT NULL CHECK(FareAmount > 0), BookingDate DATETIME DEFAULT
  CURRENT_TIMESTAMP, Status VARCHAR(20) DEFAULT 'Confirmed' CHECK(Status IN
  ('Confirmed','Cancelled','Boarded')) );
```

ALTER COLUMN — Modifying Columns in Production

Ex 3.4 Add a new column to existing Employees table

```
-- Business requirement: track LinkedIn profiles ALTER TABLE Employees ADD COLUMN
  LinkedInURL VARCHAR(200) AFTER Email; -- Add a manager reference column ALTER TABLE
  Employees ADD COLUMN ManagerID INT REFERENCES Employees(EmpID);
```

Ex 3.5 Widen a column that is too short

```
-- Email column VARCHAR(100) is too small for some corporate emails ALTER TABLE
  Employees MODIFY COLUMN Email VARCHAR(200) UNIQUE NOT NULL; -- Change Salary data type
  to support higher values ALTER TABLE Employees MODIFY COLUMN Salary DECIMAL(15,2) NOT
  NULL CHECK(Salary >= 15000);
```

4

Primary Key (PK)

Five real-world PK examples — surrogate, natural, composite, UUID, compound

A Primary Key uniquely identifies every row. Below are five examples covering the most common PK strategies used in real databases.

Ex 4.1 Surrogate Key — Auto-incrementing INT (most common)

```
CREATE TABLE Customers ( CustomerID INT PRIMARY KEY AUTO_INCREMENT, -- DB generates 1,2,3...
FullName VARCHAR(100) NOT NULL, Email VARCHAR(100) UNIQUE NOT NULL ); INSERT INTO Customers (FullName, Email) VALUES ('Anjali Nair', 'anjali@email.com'); -- CustomerID is automatically assigned: 1
INSERT INTO Customers (FullName, Email) VALUES ('Rohit Sharma', 'rohit@email.com'); -- CustomerID is automatically assigned: 2
```

Ex 4.2 Natural Key — Country code (real-world unique value)

```
CREATE TABLE Countries ( CountryCode CHAR(2) PRIMARY KEY, -- 'IN', 'US', 'UK', 'JP'
CountryName VARCHAR(80) NOT NULL UNIQUE, Currency VARCHAR(10) ); INSERT INTO Countries VALUES ('IN', 'India', 'INR'); INSERT INTO Countries VALUES ('US', 'United States', 'USD');
INSERT INTO Countries VALUES ('JP', 'Japan', 'JPY'); -- Attempting duplicate PK will FAIL:
INSERT INTO Countries VALUES ('IN', 'India (duplicate)', 'INR'); -- ERROR: Duplicate entry 'IN' for key 'PRIMARY'
```

Ex 4.3 Composite Key — Student + Course enrollment (M:N junction table)

```
CREATE TABLE Enrollments ( StudentID INT NOT NULL, CourseID INT NOT NULL, EnrolledOn DATE DEFAULT (CURRENT_DATE), Grade CHAR(2), PRIMARY KEY (StudentID, CourseID) -- BOTH columns together = unique );
INSERT INTO Enrollments (StudentID, CourseID) VALUES (1, 101); INSERT INTO Enrollments (StudentID, CourseID) VALUES (1, 102); -- same student, diff course OK
INSERT INTO Enrollments (StudentID, CourseID) VALUES (2, 101); -- diff student, same course OK -- This will FAIL (student 1 already in course 101):
INSERT INTO Enrollments (StudentID, CourseID) VALUES (1, 101); -- ERROR: Duplicate entry '1-101' for key 'PRIMARY'
```

Ex 4.4 UUID Primary Key — Distributed system (API-safe ID)

```
-- PostgreSQL: UUID generated by the database CREATE TABLE Sessions ( SessionID UUID PRIMARY KEY DEFAULT gen_random_uuid(), UserID INT NOT NULL, CreatedAt TIMESTAMPTZ DEFAULT NOW(), ExpiresAt TIMESTAMPTZ );
INSERT INTO Sessions (UserID, ExpiresAt) VALUES (42, NOW() + INTERVAL '24 hours'); -- SessionID auto-set to e.g. 'f47ac10b-58cc-4372-a567-0e02b2c3d479'
-- MySQL 8+ CREATE TABLE ApiTokens ( TokenID VARCHAR(36) PRIMARY KEY DEFAULT (UUID()), UserID INT NOT NULL );
```

5

Foreign Key (FK)

Linking tables and enforcing referential integrity with cascade actions

A Foreign Key ensures only valid, existing values are stored in the referencing column. These examples show FK enforcement and cascade behaviour.

Ex 5.1 Department → Employee (1:N) — RESTRICT delete

```
CREATE TABLE Departments ( DeptID INT PRIMARY KEY AUTO_INCREMENT, DeptName VARCHAR(80) NOT NULL UNIQUE ); CREATE TABLE Employees ( EmpID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT NULL, DeptID INT, CONSTRAINT fk_emp_dept FOREIGN KEY (DeptID) REFERENCES Departments(DeptID) ON DELETE RESTRICT -- Cannot delete a Dept while Employees exist in it ON UPDATE CASCADE -- If DeptID changes, auto-update in Employees ); -- Test RESTRICT: INSERT INTO Departments VALUES (1, 'Engineering'); INSERT INTO Employees (Name, DeptID) VALUES ('Arjun', 1); DELETE FROM Departments WHERE DeptID = 1; -- ERROR: Cannot delete – foreign key constraint fk_emp_dept
```

Ex 5.2 Customer → Orders (1:N) — CASCADE delete

```
CREATE TABLE Customers ( CustomerID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT NULL ); CREATE TABLE Orders ( OrderID INT PRIMARY KEY AUTO_INCREMENT, CustomerID INT NOT NULL, Total DECIMAL(10,2), CONSTRAINT fk_order_cust FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE CASCADE -- Delete customer → ALL their orders deleted too ); INSERT INTO Customers VALUES (1, 'Priya Nair'); INSERT INTO Orders (CustomerID, Total) VALUES (1, 2499.00), (1, 599.00); -- Delete the customer: DELETE FROM Customers WHERE CustomerID = 1; -- Both orders for CustomerID=1 are AUTOMATICALLY deleted
```

Ex 5.3 Student → Dept (1:N) — SET NULL on delete

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT NULL, DeptID INT, -- nullable: student can exist without dept CONSTRAINT fk_stu_dept FOREIGN KEY (DeptID) REFERENCES Departments(DeptID) ON DELETE SET NULL -- If dept is deleted, student's DeptID becomes NULL ); INSERT INTO Departments VALUES (3, 'Mathematics'); INSERT INTO Students (Name, DeptID) VALUES ('Sneha Rao', 3); DELETE FROM Departments WHERE DeptID = 3; -- Sneha's row still exists, but DeptID is now NULL (not deleted)
```

■ FK Violation Example

INSERT INTO Employees (Name, DeptID) VALUES ('Test', 999); will fail with: ERROR: Cannot add or update a child row — DeptID 999 does not exist in Departments.

A schema defines the entire structure of a database. Below are two complete schemas showing how tables, keys, and constraints fit together.

Ex 6.1 Bank Database Schema — 4 tables

```
CREATE DATABASE BankDB; USE BankDB; CREATE TABLE Branches ( BranchID INT PRIMARY KEY
AUTO_INCREMENT, BranchName VARCHAR(80) NOT NULL, City VARCHAR(60) NOT NULL, IFSC
CHAR(11) UNIQUE NOT NULL ); CREATE TABLE Customers ( CustomerID INT PRIMARY KEY
AUTO_INCREMENT, FullName VARCHAR(100) NOT NULL, PAN CHAR(10) UNIQUE NOT NULL, -- natural
key Phone VARCHAR(15) UNIQUE NOT NULL, Email VARCHAR(100) UNIQUE, KYCDone BOOLEAN
DEFAULT FALSE ); CREATE TABLE Accounts ( AccountNo CHAR(12) PRIMARY KEY, CustomerID INT
NOT NULL, BranchID INT NOT NULL, Type ENUM('Savings','Current','FD') DEFAULT 'Savings',
Balance DECIMAL(15,2) DEFAULT 0.00 CHECK(Balance >= 0), OpenedOn DATE DEFAULT
(CURRENT_DATE), FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE
RESTRICT, FOREIGN KEY (BranchID) REFERENCES Branches(BranchID) ON DELETE RESTRICT );
CREATE TABLE Transactions ( TxnID INT PRIMARY KEY AUTO_INCREMENT, AccountNo CHAR(12) NOT
NULL, Type ENUM('Credit','Debit') NOT NULL, Amount DECIMAL(15,2) NOT NULL CHECK(Amount >
0), TxnDate DATETIME DEFAULT CURRENT_TIMESTAMP, Remarks VARCHAR(200), FOREIGN KEY
(AccountNo) REFERENCES Accounts(AccountNo) ON DELETE CASCADE );
```

Ex 6.2 College Database Schema — 5 tables

```
CREATE DATABASE CollegeDB; USE CollegeDB; CREATE TABLE Departments ( DeptID INT PRIMARY
KEY AUTO_INCREMENT, Name VARCHAR(80) NOT NULL UNIQUE ); CREATE TABLE Professors ( ProfID
INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT NULL, Email VARCHAR(100) UNIQUE
NOT NULL, DeptID INT REFERENCES Departments(DeptID) ON DELETE SET NULL ); CREATE TABLE
Students ( StudentID INT PRIMARY KEY AUTO_INCREMENT, FullName VARCHAR(100) NOT NULL,
Email VARCHAR(100) UNIQUE NOT NULL, RollNo VARCHAR(15) UNIQUE NOT NULL, DeptID INT
REFERENCES Departments(DeptID) ON DELETE SET NULL, CGPA DECIMAL(4,2) CHECK(CGPA BETWEEN
0.0 AND 10.0) ); CREATE TABLE Courses ( CourseID INT PRIMARY KEY AUTO_INCREMENT, Title
VARCHAR(150) NOT NULL, Credits TINYINT CHECK(Credits BETWEEN 1 AND 6), ProfID INT
REFERENCES Professors(ProfID) ON DELETE SET NULL ); CREATE TABLE Enrollments ( StudentID
INT NOT NULL REFERENCES Students(StudentID) ON DELETE CASCADE, CourseID INT NOT NULL
REFERENCES Courses(CourseID) ON DELETE CASCADE, Semester VARCHAR(10), Marks DECIMAL(5,2)
CHECK(Marks BETWEEN 0 AND 100), Grade CHAR(2), PRIMARY KEY (StudentID, CourseID) );
```

Indexes dramatically improve query performance. These examples show when and why to create each type of index, along with the queries they optimise.

Ex 7.1 Login query — Index on Email column

```
-- Problem: Finding a user by email on every login is slow without index -- Without
index: full scan of 5 million rows SELECT * FROM Customers WHERE Email =
'priya@email.com'; -- SLOW -- Solution: Unique index on Email CREATE UNIQUE INDEX
idx_cust_email ON Customers(Email); -- Now the same query uses index lookup: O(log n)
instead of O(n) SELECT * FROM Customers WHERE Email = 'priya@email.com'; -- FAST --
Verify index is used: EXPLAIN SELECT * FROM Customers WHERE Email = 'priya@email.com';
-- key: idx_cust_email rows: 1 type: const ← index used!
```

Ex 7.2 Product search — Composite index on Category + Price

```
-- Common query: filter by Category, then sort by Price SELECT Name, Price FROM Products
WHERE Category = 'Electronics' ORDER BY Price ASC; -- Composite index covers both the
WHERE filter and the ORDER BY CREATE INDEX idx_prod_cat_price ON Products(Category,
Price); -- Another common query benefits too: SELECT * FROM Products WHERE Category =
'Electronics' AND Price < 5000; -- Index on (Category, Price) serves both conditions in
one B-Tree scan
```

Ex 7.3 Full-text search — Course title search

```
-- LIKE '%python%' on VARCHAR does full table scan – very slow at scale SELECT * FROM
Courses WHERE Title LIKE '%Python%'; -- NO index used -- Full-text index solution
(MySQL) CREATE FULLTEXT INDEX idx_course_title ON Courses(Title, Description); -- Now
use MATCH...AGAINST for fast full-text search SELECT Title, Credits FROM Courses WHERE
MATCH(Title, Description) AGAINST ('Python programming' IN BOOLEAN MODE); -- Uses
inverted index – extremely fast even on millions of rows
```

Ex 7.4 Audit log — Index on timestamp for range queries

```
-- Log table has millions of rows – time-range queries are common CREATE TABLE AuditLog
( LogID INT PRIMARY KEY AUTO_INCREMENT, UserID INT, Action VARCHAR(100), LoggedAt
DATETIME NOT NULL ); -- Index on timestamp for fast date-range queries CREATE INDEX
idx_log_time ON AuditLog(LoggedAt); -- Query: All actions in the last 7 days SELECT *
FROM AuditLog WHERE LoggedAt >= DATE_SUB(NOW(), INTERVAL 7 DAY) ORDER BY LoggedAt DESC;
-- Without index: scans all rows. With index: direct range scan.
```

Constraints are the database's built-in quality guards. The examples below show each constraint passing (valid data) and failing (invalid data).

NOT NULL Examples

Ex 8.1 Hospital: Patient name must never be null

```
-- Column definition PatientName VARCHAR(100) NOT NULL, -- PASS: name provided INSERT
INTO Patients (PatientName, DOB) VALUES ('Ravi Kumar', '1985-03-12'); -- FAIL: name is
NULL INSERT INTO Patients (PatientName, DOB) VALUES (NULL, '1985-03-12'); -- ERROR:
Column 'PatientName' cannot be null
```

Ex 8.2 E-commerce: Product price required, weight optional

```
CREATE TABLE Products ( ProductID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(150) NOT
NULL, -- required Price DECIMAL(10,2) NOT NULL, -- required WeightKg DECIMAL(6,3), --
optional (NULL allowed) Description TEXT -- optional ); -- PASS: only required fields
INSERT INTO Products (Name, Price) VALUES ('USB Cable', 149.00); -- FAIL: Price missing
INSERT INTO Products (Name) VALUES ('Mystery Item'); -- ERROR: Field 'Price' doesn't
have a default value
```

UNIQUE Examples

Ex 8.3 Bank: Each customer has a unique PAN number

```
-- Column definition PAN CHAR(10) UNIQUE NOT NULL, -- PASS: unique PANs INSERT INTO
Customers (Name, PAN) VALUES ('Arjun Sharma', 'ABCPS1234K'); INSERT INTO Customers
(Name, PAN) VALUES ('Priya Nair', 'XYZPN5678Q'); -- FAIL: same PAN INSERT INTO Customers
(Name, PAN) VALUES ('Fraud User', 'ABCPS1234K'); -- ERROR: Duplicate entry 'ABCPS1234K'
for key 'PAN'
```

Ex 8.4 College: Composite unique — same professor cannot teach same course twice

```
CREATE TABLE CourseAssignments ( ProfID INT NOT NULL, CourseID INT NOT NULL, Semester
VARCHAR(10) NOT NULL, CONSTRAINT uq_prof_course_sem UNIQUE (ProfID, CourseID, Semester)
); -- PASS: different semester INSERT INTO CourseAssignments VALUES (5, 101,
'2024-Odd'); INSERT INTO CourseAssignments VALUES (5, 101, '2024-Even'); -- OK,
different sem -- FAIL: exact duplicate INSERT INTO CourseAssignments VALUES (5, 101,
'2024-Odd'); -- ERROR: Duplicate entry '5-101-2024-Odd' for key 'uq_prof_course_sem'
```

CHECK Examples

Ex 8.5 HR: Employee age must be 18–65

```
Age INT CHECK(Age BETWEEN 18 AND 65), -- PASS INSERT INTO Employees (Name, Age) VALUES ('Kiran', 28); -- FAIL: underage INSERT INTO Employees (Name, Age) VALUES ('Minor', 15); -- ERROR: CHECK constraint 'Age' is violated -- FAIL: too old INSERT INTO Employees (Name, Age) VALUES ('Elder', 70); -- ERROR: CHECK constraint 'Age' is violated
```

Ex 8.6 Flights: Return date must be after departure

```
CREATE TABLE Bookings ( BookingID INT PRIMARY KEY AUTO_INCREMENT, DeptDate DATE NOT NULL, ReturnDate DATE, CONSTRAINT chk_dates CHECK(ReturnDate IS NULL OR ReturnDate > DeptDate) ); -- PASS: valid one-way INSERT INTO Bookings (DeptDate) VALUES ('2024-12-01'); -- PASS: valid return flight INSERT INTO Bookings (DeptDate, ReturnDate) VALUES ('2024-12-01', '2024-12-10'); -- FAIL: return before departure INSERT INTO Bookings (DeptDate, ReturnDate) VALUES ('2024-12-10', '2024-12-01'); -- ERROR: CHECK constraint 'chk_dates' is violated
```

DEFAULT Examples

Ex 8.7 Order management: Status defaults to Pending

```
Status VARCHAR(20) DEFAULT 'Pending', OrderDate DATE DEFAULT (CURRENT_DATE), -- Without specifying Status or OrderDate: INSERT INTO Orders (CustomerID, ProductID, Quantity) VALUES (201, 101, 2); -- Row stored as: Status='Pending', OrderDate=today's date -- Explicitly override the default: INSERT INTO Orders (CustomerID, ProductID, Quantity, Status) VALUES (202, 103, 1, 'Expedited'); -- Status='Expedited' (override applied)
```

Ex 8.8 Student registration: CGPA defaults to 0.00, active defaults to TRUE

```
CREATE TABLE Students ( StudentID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT NULL, CGPA DECIMAL(4,2) DEFAULT 0.00, IsActive BOOLEAN DEFAULT TRUE, JoinedOn DATE DEFAULT (CURRENT_DATE) ); -- New student – CGPA, IsActive, JoinedOn all use defaults INSERT INTO Students (Name) VALUES ('Deepa Krishnan'); -- Row: Name='Deepa Krishnan', CGPA=0.00, IsActive=TRUE, JoinedOn=today SELECT * FROM Students WHERE Name='Deepa Krishnan'; -- Output: 1 | Deepa Krishnan | 0.00 | 1 | 2024-06-12
```

Relationships connect tables through Foreign Keys. These three examples walk through each relationship type — table creation, data insertion, and JOIN queries.

One-to-One (1:1) — Employee and Passport

Ex 9.1 Each employee has exactly one passport record

EmpID	Name	Department
1	Arjun Sharma	Engineering
2	Priya Nair	Finance
3	Rahul Verma	HR

Passport ID	EmpID (FK UNIQUE)	PassportNo	Expiry	Country
P01	1	K1234567	2030-05-10	India
P02	2	B9876543	2029-03-22	India
P03	3	J4561230	2028-11-15	India

```
CREATE TABLE Passports ( PassportID INT PRIMARY KEY AUTO_INCREMENT, EmpID INT UNIQUE NOT NULL, -- UNIQUE enforces the 1:1
PassportNo CHAR(8) UNIQUE NOT NULL, Expiry DATE, Country VARCHAR(50), FOREIGN KEY (EmpID) REFERENCES Employees(EmpID) ON DELETE CASCADE );
-- Retrieve employee with their passport: SELECT e.Name, e.Department, p.PassportNo, p.Expiry FROM Employees e LEFT JOIN Passports p ON p.EmpID = e.EmpID;
```

■ OUTPUT

Name	Department	PassportNo	Expiry
Arjun Sharma	Engineering	K1234567	2030-05-10
Priya Nair	Finance	B9876543	2029-03-22
Rahul Verma	HR	J4561230	2028-11-15

One-to-Many (1:N) — Customer and Orders

Ex 9.2 One customer places many orders

CustomerID	Name	City
1	Anjali Nair	Mumbai
2	Rohit Singh	Delhi
3	Meena Kumar	Chennai

OrderID	CustomerID (FK)	Product	Amount	Status
101	1	Laptop	65000	Delivered

102	1	Mouse	699	Shipped
103	2	Keyboard	1499	Pending
104	1	Webcam	2199	Delivered
105	3	Monitor	18999	Delivered

```
-- Count and total of each customer's orders
SELECT c.Name, COUNT(o.OrderID) AS TotalOrders, SUM(o.Amount) AS TotalSpent
FROM Customers c LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.Name ORDER BY TotalSpent DESC;
```

■ OUTPUT

Name	TotalOrders	TotalSpent
Anjali Nair	3	67898.00
Meena Kumar	1	18999.00
Rohit Singh	1	1499.00

Many-to-Many (M:N) — Student and Courses via Enrollments

Ex 9.3 Students enrol in many courses; each course has many students

StudentID	Name	CourseID	Title	StudentID	CourseID	Grade
1	Arjun	C01	Data Structures	1	C01	A
2	Priya	C02	DBMS	1	C02	B+
3	Sneha	C03	Networks	2	C01	A+
				2	C03	B
				3	C02	A
				3	C03	A-

```
-- All courses Arjun (StudentID=1) is enrolled in:
SELECT s.Name, c.Title, e.Grade FROM Students s JOIN Enrollments e ON e.StudentID = s.StudentID
JOIN Courses c ON c.CourseID = e.CourseID WHERE s.StudentID = 1;
-- All students enrolled in DBMS (CourseID=C02):
SELECT c.Title, s.Name, e.Grade FROM Courses c JOIN Enrollments e ON e.CourseID = c.CourseID
JOIN Students s ON s.StudentID = e.StudentID WHERE c.CourseID = 'C02';
```

■ OUTPUT

Name	Title	Grade
Arjun	Data Structures	A
Arjun	DBMS	B+

ACID Properties

Bank transfer · Ticket booking · Hospital record — full transaction examples

ACID guarantees that transactions are reliable. Each example below demonstrates one or more ACID properties with a realistic real-world scenario.

Atomicity + Durability — Bank Transfer

Ex 10.1 Transfer Rs.10,000 from Savings to Current account

AccountNo	HolderName	Type	Balance
ACC001001	Arjun Sharma	Savings	85000.00
ACC001002	Arjun Sharma	Current	12000.00

```
START TRANSACTION; -- begin atomic unit -- Step 1: Debit savings account UPDATE Accounts
SET Balance = Balance - 10000 WHERE AccountNo = 'ACC001001'; -- Balance: 85000 → 75000
-- Step 2: Credit current account UPDATE Accounts SET Balance = Balance + 10000 WHERE
AccountNo = 'ACC001002'; -- Balance: 12000 → 22000 -- Step 3: Log the transaction INSERT
INTO Transactions (AccountNo, Type, Amount, Remarks) VALUES ('ACC001001', 'Debit',
10000, 'Transfer to Current'), ('ACC001002', 'Credit', 10000, 'Transfer from Savings');
COMMIT; -- All 3 steps saved permanently (Durability) -- If ANY step fails (e.g. network
error after Step 1): -- ROLLBACK; -- Balance returns to 85000 and 12000 (Atomicity) --
Neither account is partially updated
```

■ OUTPUT

After COMMIT – Accounts table:

ACC001001	Arjun Sharma	Savings	75000.00
ACC001002	Arjun Sharma	Current	22000.00

Isolation — Concurrent Ticket Booking

Ex 10.2 Two users try to book the last train seat simultaneously

[illegible]

Consistency + SAVEPOINT — Hospital Record Update

Ex 10.3 Update patient diagnosis, prescription, and billing with partial rollback

```
START TRANSACTION; -- Step 1: Update diagnosis UPDATE Patients SET Diagnosis = 'Type 2
Diabetes', LastUpdated = NOW() WHERE PatientID = 'P001'; SAVEPOINT after_diagnosis; --
safe point -- Step 2: Insert prescription INSERT INTO Prescriptions (PatientID,
Medicine, Dosage, Days) VALUES ('P001', 'Metformin 500mg', 'Twice daily', 30); SAVEPOINT
after_prescription; -- safe point -- Step 3: Generate billing record INSERT INTO Bills
(PatientID, Amount, BillDate) VALUES ('P001', 2500.00, CURRENT_DATE); -- Suppose billing
system fails at this point... ROLLBACK TO SAVEPOINT after_prescription; -- Diagnosis +
Prescription kept; billing entry undone -- Now retry billing with corrected amount:
INSERT INTO Bills (PatientID, Amount, BillDate) VALUES ('P001', 2200.00, CURRENT_DATE);
-- corrected amount COMMIT; -- All three steps (with corrected bill) saved permanently
```

ACID Summary — Quick Reference

Property	SQL Used	What It Prevented	Real Scenario
Atomicity	ROLLBACK	Debit without credit	Bank transfer partial failure
Consistency	CHECK, FK	Negative balance	Balance < 0 rejected by constraint
Isolation	FOR UPDATE lock	Double-booking	Two users booking last seat
Durability	COMMIT + WAL	Data loss on crash	Payment confirmed → survives reboot